



Orviohub Identity and Onboarding PRD

Product: Orviohub platform

Initial application: `inventory.orviohub.com`

Future applications: `taskmanagement.orviohub.com`, `gym.orviohub.com`, `booking.orviohub.com`, and others

Backend: Node.js with Fastify REST API

Database: Convex

Authentication model: Centralized Zoho-style identity and onboarding

Document version: 1.0

Date: 23 August 2026

1. Executive summary

Orviohub should use one centralized identity platform for all current and future applications.

Users should create one Orviohub account at a central authentication domain, then access enabled products from each product subdomain. A user who signs up through Inventory should later be able to access Task Management without creating a second account.

Recommended structure:

<code>orviohub.com</code>	Marketing website
<code>accounts.orviohub.com</code>	Authentication and account center
<code>app.orviohub.com</code>	Product launcher
<code>inventory.orviohub.com</code>	Inventory application
<code>taskmanagement.orviohub.com</code>	Task-management application
<code>api.orviohub.com</code>	Central REST API

The most important architectural decision is:

Authentication should belong to Orviohub, while authorization should be evaluated for each product and workspace.

Convex can be used as the application database and can integrate with external OpenID Connect providers, but the Fastify API must remain the authoritative enforcement point for REST requests. Convex documentation supports custom OIDC authentication and authentication checks inside backend functions. ^[1] ^[2]

2. Product objectives

Primary objectives

- Provide one Orviohub account across all applications.
- Support email/password authentication.
- Support Google authentication.
- Support Facebook authentication.
- Support email verification.
- Support password reset.
- Support future MFA.
- Allow users to create or join workspaces.
- Allow users to access multiple workspaces.
- Allow users to use different roles in different products.
- Provide centralized account settings.
- Provide application-specific onboarding.
- Support future subdomains without duplicating identity logic.
- Record security and onboarding events.
- Notify users about important account activity.
- Preserve a clean separation between identity and product data.

Secondary objectives

- Reduce registration friction.
- Make first-time setup understandable to nontechnical Nigerian business users.
- Allow a user to enter Inventory and later activate Task Management using the same account.
- Allow future products to reuse shared onboarding components.
- Provide support teams with safe, auditable user-assistance tools.

Non-goals

This system will not initially include:

- Enterprise SSO.
- SAML.
- SCIM provisioning.
- Advanced workforce identity management.
- Hardware security-key support.

- Complex delegated administration.
- Full multi-region identity replication.
- Anonymous business operations.
- Separate authentication databases for individual applications.

3. Recommended domain architecture

Domain structure

orviohub.com	
└─ /	Marketing landing page
└─ /products	Product directory
└─ /pricing	Pricing
└─ /resources	Help and documentation
└─ /contact	Contact and support
└─ /security	Security information
accounts.orviohub.com	
└─ /login	Sign-in
└─ /signup	Account creation
└─ /verify-email	Email verification
└─ /forgot-password	Password reset request
└─ /reset-password	Password reset completion
└─ /account	Personal account center
└─ /security	Security settings
└─ /sessions	Active sessions
└─ /connected-accounts	Google/Facebook connections
└─ /logout	Sign-out
app.orviohub.com	
└─ /	Product launcher
└─ /workspaces	Workspace selector
└─ /settings	Shared settings
└─ /billing	Subscription center
inventory.orviohub.com	
└─ /onboarding	Inventory setup
└─ /dashboard	Inventory dashboard
└─ /sell	POS
└─ /products	Product catalogue
└─ /stock	Inventory
└─ /purchases	Purchases
└─ /customers	Customers
└─ /suppliers	Suppliers
└─ /reports	Reports
taskmanagement.orviohub.com	
└─ /onboarding	Task setup
└─ /dashboard	Task dashboard
└─ /projects	Task features
api.orviohub.com	

```
|— /v1/auth
|— /v1/users
|— /v1/workspaces
|— /v1/memberships
|— /v1/products
|— /v1/inventory
|— /v1/tasks
```

Why use a central accounts domain?

A central authentication domain provides:

- One identity across all applications.
- One password-reset flow.
- One security center.
- One connected-provider system.
- Consistent session behavior.
- Easier future application launches.
- Centralized audit logs.
- Easier account deletion and export.
- Lower risk of inconsistent authentication implementations.

Each product subdomain should consume the centralized identity session rather than implement its own login screen and user database.

4. Identity architecture

Core concepts

User

A person who has an Orvionhub account.

Identity

A login method connected to a user, such as:

- Email/password.
- Google.
- Facebook.
- Future phone OTP.
- Future Apple login.
- Future enterprise SSO.

Workspace

A business or team environment where work happens.

For Inventory, a workspace may represent:

- A provision store.
- A boutique.
- A supermarket.
- A pharmacy.
- A wholesaler.

For Task Management, the same workspace may represent:

- A company.
- A department.
- A project team.
- A personal workspace.

Product

An Orvionhub application, such as:

- Inventory.
- Task Management.
- Gym.
- Bookings.
- CRM.

Workspace membership

The relationship between a user and a workspace.

A membership contains:

- User.
- Workspace.
- Status.
- Role.
- Product permissions.
- Invitation metadata.
- Created date.

Product entitlement

Whether a workspace can use a particular product.

A workspace may have:

- Inventory enabled.
- Task Management disabled.
- Gym enabled.
- Trial entitlement.
- Paid entitlement.
- Suspended entitlement.

Session

A trusted login session representing a user's authenticated device or browser.

Audit event

A recorded security, account, workspace, or application action.

5. Identity versus authorization

Authentication answers:

Who is this user?

Authorization answers:

What can this user do in this workspace and application?

These must be separate.

Example:

- User Code X has one Orvionhub identity.
- Code X belongs to Workspace A as Owner.
- Code X belongs to Workspace B as Staff.
- Code X can use Inventory in Workspace A.
- Code X can use Task Management in Workspace B.
- Code X cannot view Workspace A's data while operating in Workspace B.

The REST API should never authorize access based only on a user ID. It must evaluate:

```
user
+ workspace
+ product
```

- + membership
- + role
- + permission
- + resource ownership

6. Authentication methods

6.1 Email and password

Email/password should be available as the default universal method.

Signup fields

- First name.
- Last name.
- Email.
- Password.
- Password confirmation.
- Country.
- Acceptance of Terms.
- Acceptance of Privacy Policy.
- Optional marketing consent.

Password requirements

- Minimum length.
- Password strength validation.
- Common-password rejection.
- Password confirmation.
- Password breach checking where feasible.
- No password stored in plaintext.

Recommended behavior

After signup:

1. Create a pending user.
2. Send verification email.
3. Create a limited session or require verification before product activation.
4. Allow the user to complete basic profile details.
5. Require verification before sensitive actions.

6. Activate onboarding after verification.

6.2 Google authentication

Google should be implemented using OAuth 2.0 Authorization Code Flow with OpenID Connect.

Google's OpenID Connect documentation defines its authorization endpoint, token endpoint, discovery document, and identity claims. [\[3\]](#) [\[4\]](#)

Recommended flow

1. User selects "Continue with Google."
2. Orvionhub generates a state value.
3. Orvionhub generates a nonce.
4. User is redirected to Google.
5. Google authenticates the user.
6. Google redirects to Orvionhub callback.
7. Orvionhub exchanges the authorization code server-side.
8. Orvionhub validates:
 - Issuer.
 - Audience.
 - Signature.
 - Expiry.
 - Nonce.
 - State.
9. Orvionhub finds or creates the user.
10. Orvionhub links the Google identity.
11. Orvionhub creates an Orvionhub session.
12. User is returned to the original product or onboarding flow.

Important rules

- Do not trust a client-submitted Google profile.
- Validate the ID token server-side.
- Use the stable provider subject as the external identity key.
- Do not use email alone as the permanent provider identity.
- Require account-linking confirmation if the email already belongs to another login method.
- Store only necessary profile data.

6.3 Facebook authentication

Facebook Login should be implemented through the official OAuth flow and handled by the backend.

Meta's authentication documentation describes OAuth-based Facebook Login and access-token handling. ^[5]

Recommended flow

1. User selects "Continue with Facebook."
2. Orviohub creates state and nonce values.
3. User is redirected to Facebook.
4. Facebook authenticates the user.
5. Facebook redirects to the Orviohub callback.
6. Fastify exchanges and validates the result.
7. Orviohub retrieves permitted identity information.
8. Orviohub finds or creates the user.
9. Orviohub links the Facebook identity.
10. Orviohub creates an Orviohub session.
11. User continues onboarding or enters the requested product.

Important rules

- Request only the minimum scopes.
- Do not assume Facebook always returns a usable email.
- If no email is available, ask the user to add and verify one.
- Do not automatically merge accounts based only on name.
- Require confirmation before linking to an existing account.
- Handle provider deauthorization.
- Provide a connected-account removal flow.

6.4 Future authentication methods

The identity architecture should later support:

- Phone number OTP.
- Magic-link login.
- Apple login.
- Microsoft login.
- GitHub login.

- Passkeys.
- TOTP authenticator apps.
- Hardware security keys.
- SSO for larger organizations.
- SAML.
- OIDC enterprise providers.

These should be additional identity providers attached to the same user, not separate accounts.

7. Account linking

Problem

A user may sign up with Google and later try to sign in with email/password. Or they may use Facebook first and Google later.

Without account linking, duplicate accounts may be created.

Linking rules

Safe automatic linking

Automatic linking may occur only when:

- The provider returns a verified email.
- The email exactly matches an existing verified Orvionhub email.
- The user is currently authenticated.
- The user confirms the link if required by risk policy.

Unsafe linking

Do not automatically link based only on:

- Name.
- Phone number without verification.
- Unverified provider email.
- Similar email addresses.
- Facebook display name.
- Browser or device identity.

Account-linking flow

1. User signs in with a new provider.
2. System discovers an existing account.
3. System displays:
 - Existing email.
 - Existing login methods.
4. System asks the user to authenticate with the existing account.
5. System links the new provider.
6. System records an audit event.
7. System sends a security notification.

Connected accounts screen

The user should see:

- Google connected or not connected.
- Facebook connected or not connected.
- Email/password enabled or not enabled.
- Date connected.
- Last used date.
- Remove button.
- Warning if removing the only login method.

A user must always retain at least one usable authentication method.

8. Session architecture

Recommended model

Use centralized Orviah sessions rather than separate product-specific login sessions.

A session should contain or reference:

- Session ID.
- User ID.
- Device ID.
- Browser metadata.
- IP metadata.
- Created time.
- Last active time.

- Expiry time.
- Revoked time.
- Authentication strength.
- MFA status in future.

Browser strategy

A practical approach is:

- Use a secure, HttpOnly, SameSite cookie.
- Scope the cookie to `.orv.iohub.com` only if the central authentication model requires cross-subdomain access.
- Use HTTPS everywhere.
- Use short-lived access sessions.
- Use secure refresh-token rotation.
- Prevent JavaScript access to refresh credentials.

Because all product subdomains share the parent domain, cookie scope must be designed carefully. A compromised subdomain can create significant risk if sensitive authentication cookies are broadly available.

More secure alternative

Use a central authorization-code redirect pattern:

1. User visits `inventory.orv.iohub.com`.
2. Inventory checks for a local product session.
3. If absent, redirect to `accounts.orv.iohub.com`.
4. Accounts verifies the global session.
5. Accounts returns a short-lived one-time authorization code.
6. Inventory backend exchanges the code with the identity service.
7. Inventory creates a product-local session.

This reduces the need to expose a long-lived global token to every product subdomain.

Recommendation

Use:

- Central identity at `accounts.orv.iohub.com`.
- One-time authorization code for product handoff.
- Product-local secure session.
- Central logout event.
- Server-to-server token exchange.

- No access tokens in local storage.

This gives you centralized identity while reducing cross-subdomain credential exposure.

9. Authentication screens

Public marketing screens

1. Main landing page

URL: `orv.io/hub.com`

Sections:

- Header.
- Hero statement.
- Product overview.
- Inventory product preview.
- Future product ecosystem.
- Benefits.
- How it works.
- Pricing preview.
- Customer trust section.
- FAQ.
- Call to action.
- Footer.

Primary actions:

- Get started.
- Sign in.
- Explore Inventory.
- View pricing.

2. Products page

URL: `orv.io/hub.com/products`

Shows:

- Inventory.
- Task Management.
- Gym.

- Bookings.
- Future products.

Each product includes:

- Description.
- Target users.
- Key capabilities.
- Availability status.
- Call to action.

3. Inventory landing page

URL: orviohub.com/inventory

Content:

- Inventory problem.
- Product benefits.
- Stock tracking.
- Sales.
- Supplier management.
- Customer credit.
- Reports.
- Offline capability.
- Pricing.
- FAQ.
- Sign-up CTA.

4. Pricing page

URL: orviohub.com/pricing

Shows:

- Free trial.
- Starter.
- Growth.
- Business.
- Feature comparison.
- Usage limits.
- Billing frequency.

- FAQ.
- Contact sales or support.

5. Security page

URL: orviohub.com/security

Shows:

- Account security.
- Data protection.
- Backups.
- Access controls.
- Audit logs.
- Support access.
- Responsible disclosure contact.

6. Help center

URL: orviohub.com/help

Shows:

- Getting started.
- Account management.
- Inventory guides.
- Billing.
- Troubleshooting.
- Contact support.

Authentication screens

7. Sign-in selection

URL: accounts.orviohub.com/login

Components:

- Orviohub logo.
- Email field.
- Continue button.
- Continue with Google.
- Continue with Facebook.

- Password login option.
- Forgot password.
- Create account.
- Privacy and Terms links.

Suggested copy:

 Sign in to your Orviohub account.

The screen should remember the product the user attempted to access.

8. Email entry screen

The user enters an email first.

Why this is useful:

- Detect whether the user already has an account.
- Show the correct next step.
- Reduce confusion between signup and login.
- Support account linking.
- Prevent duplicate accounts.

Flow:

1. Enter email.
2. System checks whether account exists.
3. Existing user sees sign-in options.
4. New user sees account-creation options.

Do not reveal excessive account information to unauthenticated users. Use privacy-safe messaging.

9. Password screen

Components:

- Email summary.
- Password field.
- Show/hide password.
- Forgot password.
- Sign in.
- Back button.
- Rate-limit messaging.

10. Google redirect and callback

Users normally should not see a separate callback screen, but the system should have:

- Loading state.
- Authentication failure state.
- Cancelled-flow state.
- Account-linking state.
- Provider-error state.

11. Facebook redirect and callback

Same states as Google:

- Loading.
- Failed.
- Cancelled.
- Missing email.
- Account-linking required.
- Provider unavailable.

12. Sign-up screen

Components:

- First name.
- Last name.
- Email.
- Password.
- Confirm password.
- Country.
- Terms checkbox.
- Privacy checkbox.
- Marketing consent.
- Create account.
- Google and Facebook options.

13. Email verification screen

States:

- Verification email sent.
- Resend email.
- Change email.
- Verification successful.
- Link expired.
- Link already used.
- Too many resend attempts.

Suggested copy:

 Check your email to verify your Orvionhub account.

14. Password reset request

Components:

- Email field.
- Submit button.
- Back to login.
- Security-safe success message.

Do not reveal whether an email exists.

15. Password reset completion

Components:

- New password.
- Confirm password.
- Password requirements.
- Success message.
- Sign-in button.

After reset:

- Revoke other sessions.
- Send security notification.
- Require login again.

16. Authentication error page

Possible errors:

- Invalid or expired callback.
- Provider denied access.
- Session expired.
- Link expired.
- Account suspended.
- Unsupported browser.
- Temporary service error.

Every error page should provide:

- Explanation.
- Retry action.
- Return to login.
- Support link.
- Error reference ID.

10. Onboarding architecture

Onboarding should have two layers:

Layer 1: Global account onboarding

This is shared by every Orviohub application.

It establishes:

- User identity.
- Name.
- Profile.
- Country.
- Time zone.
- Communication preferences.
- Basic account security.
- Workspace choice.

Layer 2: Product onboarding

Each application has its own onboarding.

Inventory onboarding establishes:

- Business type.
- Workspace.
- Store or branch.
- Currency.
- Products.
- Opening stock.
- Staff.
- Receipt settings.
- First sale.

Task Management onboarding may establish:

- Team or personal workspace.
- Work style.
- First project.
- Members.
- Task preferences.

The shared onboarding engine should provide reusable components, while each product defines its own questions and completion rules.

11. Global onboarding screens

11.1 Welcome screen

URL: `accounts.orviohub.com/welcome`

Content:

 Welcome to Orviohub.

Actions:

- Create a workspace.
- Join an existing workspace.
- Explore products.
- Skip for now.

11.2 Profile setup

Fields:

- First name.
- Last name.
- Profile photo.
- Country.
- State or region.
- Time zone.
- Preferred language.
- Phone number, optional initially.

The user should not be forced to fill unnecessary fields before accessing the product.

11.3 Workspace selection

Possible choices:

- Create a new workspace.
- Join through invitation.
- Continue to personal workspace.
- Switch workspace.

11.4 Create workspace screen

Fields:

- Workspace name.
- Workspace type.
- Country.
- State.
- City.
- Phone number.
- Logo, optional.
- Time zone.
- Default currency.

The same screen can adapt to different products.

11.5 Join workspace screen

Users can join through:

- Invitation link.
- Invitation email.
- Workspace code, if later implemented.
- Admin approval.

Display:

- Workspace name.
- Inviting person.
- Product.
- Intended role.
- Invitation expiry.
- Accept or decline.

11.6 Product selection screen

Show products:

- Inventory.
- Task Management.
- Gym.
- Bookings.
- CRM.

Each product should display:

- Short description.
- Availability.
- Free trial status.
- Required workspace type.
- Activate button.

11.7 Workspace confirmation

Before entering the product, show:

- Workspace name.
- Product.
- Role.
- Trial or subscription status.

- Next setup steps.

12. Inventory onboarding journey

Entry points

Users can enter Inventory onboarding from:

- Inventory landing page.
- Product directory.
- Product subdomain.
- App launcher.
- Workspace dashboard.
- Invitation link.

Inventory onboarding screens

1. Inventory welcome

Content:

 Let's set up your inventory workspace.

Show setup steps:

- Business details.
- Store setup.
- Products.
- Opening stock.
- Staff.
- First sale.

2. Business type

Options:

- Provision store.
- Supermarket.
- Boutique.
- Cosmetics.
- Electronics.
- Pharmacy.

- Wholesale.
- Other.

This choice can personalize defaults but must not lock the user into an inflexible workflow.

3. Business profile

Fields:

- Business name.
- Phone number.
- Address.
- City.
- State.
- Logo.
- Receipt name.
- Tax identification fields, optional.

4. Store or branch setup

Fields:

- Store name.
- Store address.
- Store phone.
- Manager.
- Default location.
- Receipt settings.

For MVP, create one primary branch automatically.

5. Currency and pricing

Fields:

- Currency: NGN by default.
- Prices include tax: Yes/No.
- Default retail price.
- Wholesale price enabled: Yes/No.
- Negative stock allowed: No by default.

6. Product setup choice

Choices:

- Add products manually.
- Import products from CSV.
- Start with sample products.
- Skip and add later.

7. Manual product creation

Fields:

- Product name.
- SKU.
- Barcode.
- Category.
- Unit.
- Cost price.
- Selling price.
- Opening quantity.
- Reorder level.

8. CSV import

Steps:

1. Download template.
2. Upload CSV.
3. Validate.
4. Review errors.
5. Confirm import.
6. Show completion summary.

9. Opening stock

Options:

- Enter opening stock manually.
- Import opening stock.
- Start with zero.
- Add later.

10. Staff invitation

Fields:

- Staff name.
- Email or phone.
- Role.
- Branch.
- Product access.

Actions:

- Invite.
- Skip.
- Add later.

11. Receipt settings

Fields:

- Receipt header.
- Footer.
- Logo.
- Phone number.
- Address.
- Return policy.
- Tax display.
- Print settings.

12. First sale tutorial

The system guides the user through:

1. Select product.
2. Add quantity.
3. Select payment method.
4. Complete sale.
5. View receipt.
6. View stock reduction.

13. Onboarding completion

Show:

- Products added.
- Opening stock added.
- Staff invited.
- First sale completed.
- Suggested next actions.

Actions:

- Go to dashboard.
- Add more products.
- Set reorder levels.
- Explore reports.
- Invite staff.

13. Onboarding progress model

Each onboarding flow should have:

- Flow ID.
- User ID.
- Workspace ID.
- Product ID.
- Current step.
- Completed steps.
- Skipped steps.
- Required steps.
- Started date.
- Last updated date.
- Completed date.
- Version.

Example:

```
{
  "flow": "inventory_setup",
  "workspaceId": "workspace_123",
  "userId": "user_456",
  "currentStep": "opening_stock",
  "completedSteps": [
```

```
    "business_profile",
    "branch_setup",
    "currency_setup"
  ],
  "skippedSteps": [
    "staff_invitation"
  ],
  "status": "in_progress"
}
```

Onboarding rules

- Users can leave and resume.
- Completed steps should not be repeated unnecessarily.
- Skipped steps remain available later.
- Required security steps cannot be skipped.
- Product-specific steps can be completed after dashboard access.
- Users should not lose progress if onboarding fails.
- Onboarding events should be audited.
- Users can restart only where safe.

14. Dashboard entry behavior

After authentication, determine the user's destination.

Routing logic

```
If user is unauthenticated:
  redirect to accounts.orviohub.com/login

If user is authenticated but has no workspace:
  redirect to global onboarding

If user has been invited to a workspace:
  show invitation decision

If user selected an application with incomplete setup:
  redirect to application onboarding

If user has access and setup is complete:
  redirect to application dashboard

If user lacks access:
  show access request or product activation screen
```

Example

User visits:

```
https://inventory.orviohub.com
```

System checks:

1. Is there a valid product session?
2. Is the user authenticated?
3. Which workspace is active?
4. Does the user have Inventory entitlement?
5. Is Inventory onboarding complete?
6. What is the user's role?
7. Redirect to the correct page.

Possible destinations:

```
/accounts.orviohub.com/login  
/accounts.orviohub.com/welcome  
/app.orviohub.com/workspaces  
/inventory.orviohub.com/onboarding  
/inventory.orviohub.com/dashboard  
/app.orviohub.com/activate/inventory
```

15. Workspace and app access model

Workspace creation

A user who creates a workspace automatically becomes:

- Workspace owner.
- First administrator.
- Product administrator for any activated product.

Workspace membership

Membership statuses:

- Invited.
- Pending.
- Active.
- Suspended.
- Removed.

- Declined.

Product membership

A user may be a member of a workspace but not have access to every product.

Example:

User	Workspace	Inventory	Task Management
Owner	ABC Stores	Admin	Admin
Sales attendant	ABC Stores	Sales role	No access
Project manager	ABC Stores	No access	Admin
Accountant	ABC Stores	Accountant	No access

Product entitlements

Product access should be based on:

- Workspace product activation.
- Subscription plan.
- User membership.
- Product role.
- Account status.

16. REST API design

API base URL

```
https://api.orviohub.com/v1
```

API groups

```
/v1/auth  
/v1/users  
/v1/workspaces  
/v1/memberships  
/v1/products  
/v1/entitlements  
/v1/onboarding  
/v1/notifications  
/v1/audit-logs  
/v1/billing  
/v1/inventory  
/v1/tasks
```

Authentication endpoints

```
POST /v1/auth/signup
POST /v1/auth/login
POST /v1/auth/logout
POST /v1/auth/refresh
POST /v1/auth/verify-email
POST /v1/auth/resend-verification
POST /v1/auth/forgot-password
POST /v1/auth/reset-password
GET /v1/auth/providers/google/start
GET /v1/auth/providers/google/callback
GET /v1/auth/providers/facebook/start
GET /v1/auth/providers/facebook/callback
GET /v1/auth/session
POST /v1/auth/revoke-session
POST /v1/auth/logout-all
```

User endpoints

```
GET /v1/users/me
PATCH /v1/users/me
GET /v1/users/me/identities
POST /v1/users/me/identities/link
DELETE /v1/users/me/identities/:identityId
GET /v1/users/me/sessions
DELETE /v1/users/me/sessions/:sessionId
DELETE /v1/users/me
```

Workspace endpoints

```
GET /v1/workspaces
POST /v1/workspaces
GET /v1/workspaces/:workspaceId
PATCH /v1/workspaces/:workspaceId
POST /v1/workspaces/:workspaceId/invitations
GET /v1/workspaces/:workspaceId/members
PATCH /v1/workspaces/:workspaceId/members/:membershipId
DELETE /v1/workspaces/:workspaceId/members/:membershipId
```

Product access endpoints

```
GET /v1/products
GET /v1/workspaces/:workspaceId/products
POST /v1/workspaces/:workspaceId/products/:productKey/activate
POST /v1/workspaces/:workspaceId/products/:productKey/deactivate
GET /v1/workspaces/:workspaceId/products/:productKey/access
```

Onboarding endpoints

```
GET    /v1/onboarding
GET    /v1/onboarding/:productKey
POST   /v1/onboarding/:productKey/start
PATCH /v1/onboarding/:productKey/progress
POST   /v1/onboarding/:productKey/complete-step
POST   /v1/onboarding/:productKey/skip-step
POST   /v1/onboarding/:productKey/complete
POST   /v1/onboarding/:productKey/reset
```

Notification endpoints

```
GET    /v1/notifications
PATCH /v1/notifications/:notificationId/read
POST   /v1/notifications/read-all
GET    /v1/notification-preferences
PATCH /v1/notification-preferences
```

Audit endpoints

```
GET /v1/audit-logs
GET /v1/audit-logs/:eventId
```

Audit access must be restricted to authorized workspace administrators or platform administrators.

17. Fastify backend structure

Recommended modules:

```
src/
├── modules/
│   ├── auth/
│   ├── users/
│   ├── workspaces/
│   ├── memberships/
│   ├── products/
│   ├── entitlements/
│   ├── onboarding/
│   ├── notifications/
│   ├── audit/
│   ├── billing/
│   └── inventory/
├── plugins/
│   ├── auth.ts
│   ├── convex.ts
│   ├── logger.ts
│   └── rate-limit.ts
```



```
|   └─ security.ts
└─ middleware/
    │   └─ require-auth.ts
    │   └─ require-workspace.ts
    │   └─ require-product-access.ts
    │   └─ require-permission.ts
    └─ integrations/
        │   └─ google.ts
        │   └─ facebook.ts
        │   └─ email.ts
        │   └─ payments.ts
    └─ server.ts
```

Request context

After authentication, attach a trusted request context:

```
type RequestContext = {
  userId: string
  sessionId: string
  workspaceId?: string
  productKey?: string
  membershipId?: string
  roles: string[]
  permissions: string[]
}
```

Never accept `userId`, `workspaceId`, or role information from the client as authoritative.

Fastify hooks

Suggested request flow:

```
onRequest:
  assign request ID

preValidation:
  validate headers and body

preHandler:
  authenticate session

preHandler:
  resolve workspace

preHandler:
  check product entitlement

preHandler:
  check permission

handler:
  execute business operation
```

```
onResponse:
  record latency and outcome
```

18. Convex data model

users

```
_id
email
emailNormalized
emailVerifiedAt
firstName
lastName
displayName
avatarUrl
phone
phoneVerifiedAt
country
timezone
status
createdAt
updatedAt
lastLoginAt
```

authIdentities

```
_id
userId
provider
providerSubject
providerEmail
providerEmailVerified
createdAt
lastUsedAt
metadata
```

Provider values:

```
password
google
facebook
phone
apple
```

Create a unique index on:

```
provider + providerSubject
```

sessions

```
_id  
userId  
sessionHash  
deviceId  
deviceName  
ipAddress  
userAgent  
createdAt  
lastActiveAt  
expiresAt  
revokedAt  
authenticationMethod  
mfaVerified
```

Store a hash of the refresh token, not the raw token.

workspaces

```
_id  
name  
slug  
type  
ownerId  
country  
state  
city  
timezone  
currency  
logoUrl  
status  
createdAt  
updatedAt
```

Do not use workspace subdomains for the first architecture. The `slug` is useful for display, URLs inside a product, invitations, and API references, but not as the hostname.

workspaceMemberships

```
_id  
workspaceId  
userId  
status  
defaultRole  
invitedBy  
invitedAt  
acceptedAt  
suspendedAt  
createdAt  
updatedAt
```

products

```
_id  
key  
name  
description  
subdomain  
status  
createdAt  
updatedAt
```

Example:

```
inventory  
taskmanagement  
gym  
booking
```

workspaceProducts

```
_id  
workspaceId  
productKey  
status  
planId  
trialStartedAt  
trialEndsAt  
activatedBy  
activatedAt  
suspendedAt
```

productMemberships

```
_id  
workspaceId  
userId  
productKey  
role  
permissions  
status  
createdAt  
updatedAt
```

invitations

```
_id  
workspaceId  
productKey  
email
```

```
invitedUserId
role
tokenHash
status
invitedBy
expiresAt
acceptedAt
createdAt
```

Never store invitation tokens in plaintext.

onboardingFlows

```
_id
userId
workspaceId
productKey
flowVersion
status
currentStep
completedSteps
skippedSteps
startedAt
lastUpdatedAt
completedAt
```

onboardingEvents

```
_id
userId
workspaceId
productKey
step
eventType
metadata
createdAt
```

notifications

```
_id
userId
workspaceId
productKey
type
title
body
severity
channel
status
```

```
readAt
createdAt
```

notificationPreferences

```
_id
userId
category
channel
enabled
updatedAt
```

auditLogs

```
_id
actorUserId
targetUserId
workspaceId
productKey
eventType
entityType
entityId
severity
ipAddress
userAgent
requestId
metadata
createdAt
```

19. Authentication security requirements

OAuth security

For Google and Facebook:

- Use authorization code flow.
- Generate and validate state.
- Generate and validate nonce.
- Validate redirect URI.
- Validate token issuer.
- Validate audience.
- Validate token signature.
- Validate token expiry.
- Use provider discovery or trusted configuration.
- Store provider secrets only server-side.

- Request minimum scopes.
- Log provider-linking events.

Session security

- Use secure cookies.
- Use HttpOnly cookies.
- Use SameSite protection.
- Rotate refresh tokens.
- Revoke tokens after password reset.
- Revoke tokens after suspicious activity.
- Provide logout from one device.
- Provide logout from all devices.
- Avoid local-storage refresh tokens.
- Apply session expiration.
- Apply idle timeout for sensitive administration.

Password security

- Strong password hashing.
- Password strength feedback.
- Rate limiting.
- Credential-stuffing protection.
- Reset-token expiry.
- One-time reset tokens.
- Session revocation after reset.
- Security notification after reset.

CSRF protection

If browser cookies are used, protect state-changing requests through:

- SameSite cookies.
- CSRF tokens where necessary.
- Origin checking.
- Strict CORS configuration.
- No wildcard credentials.

CORS

Allow only known origins:

```
https://orv.iohub.com
https://accounts.orv.iohub.com
https://app.orv.iohub.com
https://inventory.orv.iohub.com
https://taskmanagement.orv.iohub.com
```

Do not use:

```
Access-Control-Allow-Origin: *
```

with credentials.

Rate limits

Apply rate limits to:

- Login.
- Signup.
- Password reset.
- Email verification resend.
- OAuth initiation.
- Invitation creation.
- Session refresh.
- Account linking.
- API requests.
- Sensitive administrative actions.

20. Notifications

Notification categories

Security

- New login.
- New device.
- Password changed.
- Password reset.
- Google account linked.

- Facebook account linked.
- Login failure threshold exceeded.
- Session revoked.
- Email changed.
- Phone changed.
- MFA enabled or disabled.

Account

- Email verification.
- Profile updated.
- Account deletion request.
- Data export ready.
- Terms or policy update.

Workspace

- Workspace created.
- Invitation sent.
- Invitation accepted.
- Invitation declined.
- User joined.
- User removed.
- User role changed.
- Workspace ownership transferred.

Product

- Inventory activated.
- Inventory onboarding completed.
- Task Management activated.
- Subscription changed.
- Product access suspended.

Operational

- Offline sync failure.
- Inventory import completed.
- Inventory import failed.
- Payment subscription failed.

- Report export ready.

Notification channels

- In-app.
- Email.
- Push notification in future.
- SMS for selected security events.
- WhatsApp only where appropriate and officially supported.

Notification preferences

Users should control:

- Marketing messages.
- Product tips.
- Daily summaries.
- Weekly summaries.
- Non-critical product notifications.
- Email frequency.

Security notifications should not be fully disableable.

Notification delivery model

Use an event-based system:

```
graph TD; A[Action occurs] --> B[Audit event created]; B --> C[Notification event generated]; C --> D[Notification worker processes event]; D --> E[Email/in-app/push delivery]; E --> F[Delivery status recorded];
```

Notification requirements

- Notifications must be idempotent.
- Failed deliveries must retry.
- Security notices must be prioritized.
- Every notification should have a delivery status.

- The user should be able to mark notifications as read.
- Critical notifications should link to the relevant security or account page.

21. Audit logs

Audit-log goals

Audit logs should help:

- Detect suspicious activity.
- Support customer service.
- Investigate account issues.
- Track workspace changes.
- Explain product actions.
- Support future compliance requirements.
- Establish accountability.

Event categories

Authentication

- `auth.signup_completed`
- `auth.login_success`
- `auth.login_failed`
- `auth.logout`
- `auth.password_reset_requested`
- `auth.password_reset_completed`
- `auth.email_verified`
- `auth.provider_linked`
- `auth.provider_unlinked`
- `auth.session_revoked`
- `auth.logout_all`

User account

- `user.profile_updated`
- `user.email_changed`
- `user.phone_changed`
- `user.account_deleted`

- `user.export_requested`

Workspace

- `workspace.created`
- `workspace.updated`
- `workspace.member_invited`
- `workspace.member_joined`
- `workspace.member_removed`
- `workspace.role_changed`
- `workspace.ownership_transferred`

Product

- `product.activated`
- `product.deactivated`
- `product.onboarding_started`
- `product.onboarding_completed`

Inventory

- `inventory.product_created`
- `inventory.stock_adjusted`
- `inventory.sale_created`
- `inventory.sale_cancelled`
- `inventory.import_completed`

Audit-log fields

- Event ID.
- Event type.
- Actor.
- Target user.
- Workspace.
- Product.
- Entity type.
- Entity ID.
- IP address.
- User agent.
- Request ID.

- Severity.
- Metadata.
- Created timestamp.

Audit-log rules

- Logs should be append-only.
- Users should not be able to delete logs.
- Sensitive metadata must be redacted.
- Passwords and tokens must never be logged.
- Logs must have retention policy.
- Logs should be searchable by authorized administrators.
- Support access must itself be audited.
- Logs should be available in product-specific views where appropriate.

User-facing security activity

Users should see a simplified activity view:

- Device.
- Approximate location.
- Date.
- Authentication method.
- Action.
- Success or failure.

Do not expose raw internal security metadata unnecessarily.

22. Account settings screens

Account overview

- Name.
- Email.
- Phone.
- Country.
- Time zone.
- Profile photo.
- Account status.

Security settings

- Change password.
- Connected accounts.
- Active sessions.
- Sign out all sessions.
- MFA in future.
- Security activity.
- Recovery options.

Connected accounts

- Google.
- Facebook.
- Email/password.
- Future providers.

Notification settings

- Email preferences.
- In-app preferences.
- Product updates.
- Marketing preferences.
- Daily summaries.

Privacy and data

- Download data.
- Request account deletion.
- View privacy policy.
- Manage consent.
- View connected applications.

Workspace settings

- Workspace name.
- Members.
- Roles.
- Products.
- Subscription.
- Billing.

- Workspace deletion or transfer.

23. Logout behavior

Normal logout

When the user logs out:

- Revoke the current product session.
- Clear local session state.
- Redirect to login or public landing page.
- Record audit event.

Logout from all devices

When selected:

- Revoke all refresh sessions.
- Invalidate product sessions.
- Preserve audit record.
- Send security notification.
- Require login on all devices.

Global logout

If a user logs out from `accounts.orviohub.com`, all active product sessions should eventually be invalidated.

Because distributed products may have local sessions, use:

- Short local session lifetime.
- Session introspection or revocation checks.
- Central revocation events.
- Back-channel logout where appropriate.

24. Account recovery

Recovery options

- Password reset through verified email.
- Recovery through a verified phone number in future.
- Support-assisted recovery with strict verification.
- Backup codes in a future MFA release.

Support recovery process

Support must not change account ownership solely because someone knows:

- Business name.
- Email address.
- Phone number.
- Workspace name.

Additional verification should be required, and all support actions must be logged.

25. Invitations

Invitation flow

1. Owner or authorized manager opens Members.
2. Selects product.
3. Enters invitee email or phone.
4. Selects role.
5. Selects workspace access.
6. Sends invitation.
7. Invitee receives email or link.
8. Invitee signs in or creates an account.
9. Invitee reviews workspace and role.
10. Invitee accepts.
11. Membership becomes active.
12. Inviter receives notification.

Invitation rules

- Invitation expires.
- Invitation can be resent.
- Invitation can be revoked.
- Existing users can accept without creating a second account.
- The invitee must not gain access before acceptance.
- Invitation role must be displayed before acceptance.
- Invitation token must be single-use.
- Invitation events must be audited.

26. Workspace switching

A user may belong to many workspaces.

Workspace switcher

Display:

- Workspace name.
- Workspace logo.
- Product access.
- Role.
- Recent workspaces.
- Search.
- Create workspace.
- Join workspace.

Switching behavior

When switching:

- Clear current product context.
- Request new authorization context.
- Refresh permissions.
- Load the new workspace.
- Do not retain data from the previous workspace in the UI.
- Record workspace-switch event where useful.

Default workspace

The user can select a default workspace per product.

Example:

- Default Inventory workspace: Code X Stores.
- Default Task Management workspace: Orvionhub Team.

27. Application launcher

app.orviohub.com

The launcher should show:

- Enabled products.
- Available products.
- Trial products.
- Recently used products.
- Workspace selector.
- Account settings.
- Notifications.
- Billing.
- Help.

Each product card should show:

- Product name.
- Description.
- Access status.
- Workspace.
- Last used date.
- Open button.
- Activate button where applicable.

Product handoff

When the user selects Inventory:

1. Launcher verifies product entitlement.
2. Launcher creates a one-time authorization code.
3. Browser redirects to `inventory.orviohub.com`.
4. Inventory backend exchanges the code.
5. Inventory creates local session.
6. User enters the correct workspace and dashboard.

28. Onboarding API and configuration model

To make onboarding reusable, define product onboarding as configuration.

Example:

```

{
  "productKey": "inventory",
  "version": "1.0",
  "steps": [
    {
      "key": "business_profile",
      "type": "form",
      "required": true
    },
    {
      "key": "branch_setup",
      "type": "form",
      "required": true
    },
    {
      "key": "product_setup",
      "type": "choice",
      "required": true
    },
    {
      "key": "opening_stock",
      "type": "form",
      "required": false
    },
    {
      "key": "staff_invitation",
      "type": "invitation",
      "required": false
    },
    {
      "key": "first_sale",
      "type": "guided_action",
      "required": false
    }
  ]
}

```

Future Task Management configuration:

```

{
  "productKey": "taskmanagement",
  "version": "1.0",
  "steps": [
    {
      "key": "workspace_profile",
      "type": "form",
      "required": true
    },
    {
      "key": "first_project",
      "type": "form",
      "required": false
    },
    {

```

```
    "key": "invite_team",  
    "type": "invitation",  
    "required": false  
  }  
]  
}
```

This gives you one onboarding engine with product-specific definitions.

29. Onboarding analytics

Track:

- Landing page viewed.
- Signup started.
- Signup completed.
- Authentication method selected.
- Email verified.
- Workspace created.
- Workspace joined.
- Product selected.
- Onboarding started.
- Onboarding step completed.
- Onboarding step skipped.
- CSV import started.
- CSV import completed.
- Staff invited.
- First sale completed.
- Onboarding completed.
- Dashboard first viewed.
- Product activated.
- Trial started.
- Subscription started.

Important funnel metrics

- Landing-to-signup conversion.
- Signup completion rate.
- Verification completion rate.
- Workspace creation rate.

- Product activation rate.
- Onboarding completion rate.
- Time to first value.
- Time to first Inventory sale.
- Percentage returning within seven days.
- Percentage inviting staff.
- Trial-to-paid conversion.

30. Error and edge-case handling

User closes onboarding

Behavior:

- Save progress.
- Return on next login.
- Show “Continue setup.”
- Allow dashboard access if mandatory steps are complete.

Email verification expires

Behavior:

- Show resend option.
- Invalidate old token.
- Rate-limit resends.
- Log the event.

OAuth user has no email

Behavior:

- Ask user for email.
- Send verification email.
- Do not activate sensitive account functions until verified.

User attempts duplicate signup

Behavior:

- Avoid revealing account existence.
- Offer general next steps.
- Allow login or password reset through a safe message.

- Offer provider sign-in.

User has invitation and existing workspace

Behavior:

- Show invitation separately.
- Do not automatically switch workspace without confirmation.

User lacks product entitlement

Behavior:

- Show product access page.
- Explain why access is unavailable.
- Offer activation or request-access action.

Workspace is suspended

Behavior:

- Show suspension reason at a high level.
- Provide support link.
- Prevent product access.
- Keep account settings available.

OAuth provider unavailable

Behavior:

- Display retry.
- Offer email/password.
- Record provider failure.
- Do not create partial accounts.

31. Privacy and data governance

The platform should define:

- What personal data is collected.
- Why it is collected.
- How long it is retained.
- Which providers receive it.
- How users export it.

- How users delete it.
- How business data is handled after account deletion.
- How support access works.
- How logs are retained.

Do not collect unnecessary data during signup. A user should not need to provide extensive business information before understanding the product.

Account deletion

Deletion should include:

1. User requests deletion.
2. System explains consequences.
3. User confirms identity.
4. System applies cooling-off period where appropriate.
5. Product data handling is explained.
6. Account is deactivated.
7. Personal data is deleted or anonymized according to policy.
8. Required audit records are retained where legally necessary.

Workspace deletion should be separate from personal account deletion.

32. Recommended user journeys

Journey A: New user from Inventory landing page

```
inventory landing page
↓
Get started
↓
accounts.orviohub.com/signup
↓
Choose email, Google, or Facebook
↓
Verify email if required
↓
Global profile setup
↓
Create workspace
↓
Inventory onboarding
↓
Business setup
↓
Product setup
↓
Opening stock
```

↓
First sale
↓
Inventory dashboard

Journey B: Existing user opens Inventory

inventory.orviohub.com
↓
No local session
↓
accounts.orviohub.com/login
↓
Successful central authentication
↓
Authorization code returned
↓
Inventory creates local session
↓
Workspace selection if necessary
↓
Inventory dashboard

Journey C: Existing user activates Task Management

app.orviohub.com
↓
Select Task Management
↓
Activate product
↓
Task onboarding
↓
Create project
↓
Invite team
↓
Task Management dashboard

Journey D: Staff member accepts invitation

Invitation link
↓
accounts.orviohub.com/invitations/:token
↓
Sign in or create account
↓
Review workspace and role
↓
Accept invitation
↓

Product authorization



Product dashboard

Journey E: User links Google after email signup

Account settings



Connected accounts



Connect Google



Google consent



Server validates provider



Google linked



Security notification

33. Implementation phases

Phase 1: Identity foundation

Build:

- User model.
- Email/password signup.
- Login.
- Logout.
- Password reset.
- Email verification.
- Central session.
- User profile.
- Basic audit logs.
- Basic notification email.

Phase 2: Social authentication

Build:

- Google OAuth/OIDC.
- Facebook Login.
- Provider callback handling.

- Account linking.
- Connected accounts.
- Provider error handling.

Phase 3: Workspace foundation

Build:

- Workspace creation.
- Workspace listing.
- Workspace switching.
- Memberships.
- Invitations.
- Roles.
- Product entitlements.

Phase 4: Reusable onboarding

Build:

- Onboarding engine.
- Progress tracking.
- Step configuration.
- Resume capability.
- Product-specific onboarding.
- Analytics events.

Phase 5: Inventory onboarding

Build:

- Business profile.
- Branch setup.
- Product setup.
- CSV import.
- Opening stock.
- Staff invitation.
- Receipt settings.
- First-sale guide.
- Dashboard handoff.

Phase 6: Security maturity

Build:

- Session management.
- Logout all devices.
- Security activity.
- Enhanced audit logs.
- Account export.
- Account deletion.
- MFA preparation.

Phase 7: Future platform support

Build:

- Application launcher.
- Task Management onboarding.
- Cross-product navigation.
- Product-specific permission systems.
- Central billing.
- Future SSO architecture.

34. Testing plan

Authentication tests

- Signup success.
- Signup validation.
- Duplicate signup.
- Email verification.
- Expired verification.
- Resend verification.
- Login success.
- Incorrect password.
- Rate limiting.
- Password reset.
- Expired reset token.
- Logout.

- Logout all sessions.

Google tests

- Successful sign-in.
- Cancelled sign-in.
- Invalid state.
- Invalid nonce.
- Expired ID token.
- Account linking.
- Provider email unavailable.
- Provider account removal.

Facebook tests

- Successful sign-in.
- Cancelled sign-in.
- Missing email.
- Invalid token.
- Account linking.
- Deauthorization.
- Provider failure.

Workspace tests

- Create workspace.
- Join workspace.
- Invitation expiry.
- Invitation revocation.
- Role assignment.
- User removal.
- Workspace switching.
- Suspended workspace.

Product access tests

- Product enabled.
- Product disabled.
- Product trial expired.
- User lacks access.

- User has restricted role.
- Correct subdomain routing.

Security tests

- Cross-tenant access attempt.
- Cross-product access attempt.
- Session replay.
- CSRF.
- CORS.
- Token leakage.
- Brute-force login.
- Privilege escalation.
- Audit-log tampering.
- Insecure direct object reference.

Onboarding tests

- Resume after closing.
- Skip optional step.
- Required step enforcement.
- Import failure.
- Partial import.
- Duplicate product.
- First sale completion.
- Product onboarding completed.
- Re-onboarding existing workspace.

35. Release acceptance criteria

The authentication and onboarding system is ready for MVP launch when:

- A user can create one Orvionhub account.
- Email/password signup works.
- Email verification works.
- Password reset works.
- Google authentication works.
- Facebook authentication works.
- Account linking prevents unnecessary duplicates.

- User can create a workspace.
- User can join a workspace.
- User can invite another user.
- Membership roles work.
- Product entitlements work.
- Inventory subdomain authentication works.
- Unauthenticated users are redirected centrally.
- Authenticated users are returned to the correct application.
- Workspace switching works.
- Onboarding progress is saved.
- Inventory onboarding ends at the correct dashboard.
- Product-level permissions are enforced by Fastify.
- Convex data is tenant-scoped.
- Security events generate audit logs.
- Security events generate notifications.
- Logout invalidates sessions.
- Logout-all works.
- Basic privacy and account controls exist.
- OAuth secrets are never exposed to the frontend.
- No authentication tokens are stored insecurely in local storage.
- Cross-workspace data access has been tested and blocked.

36. Final recommendation

Use this model:

```
One Orviohub account
  ↓
Central authentication at accounts.orviohub.com
  ↓
Shared users, workspaces, memberships, and entitlements
  ↓
One-time authorization handoff
  ↓
Product-specific sessions
  ↓
inventory.orviohub.com
taskmanagement.orviohub.com
future-app.orviohub.com
```

The most important design rule is:

Keep identity centralized, but keep product onboarding and product permissions modular.

This allows Orviohub to behave like Zoho for users while maintaining clear application boundaries. A user should authenticate once, choose a workspace, activate the desired application, complete that application's onboarding, and later enter another Orviohub application without creating another account.

For your Fastify and Convex stack, keep authentication and authorization logic behind well-defined platform services. Fastify should validate requests and enforce permissions, while Convex stores users, workspaces, memberships, onboarding progress, notifications, audit events, and application data. Convex supports custom OIDC provider integration, which makes it suitable for connecting external authentication providers while preserving your centralized Orviohub identity model. ^[2]

✱

1. <https://docs.convex.dev/auth/overview>
2. <https://docs.convex.dev/auth/advanced/custom-auth>
3. <https://developers.google.com/identity/openid-connect/reference>
4. <https://developers.google.com/identity/openid-connect/openid-connect>
5. <https://raw.githubusercontent.com/api-evangelist/meta/refs/heads/main/authentication/meta-authentication.yml>
6. <https://cloud.google.com/identity-platform/docs/web/oidc?hl=id>
7. <https://www.convex.dev/can-do/auth>
8. <https://docs.cloud.google.com/identity-platform/docs/web/oidc>
9. <https://cloud.google.com/identity-platform/docs/web/oidc?hl=pt-BR>
10. <https://docs.convex.dev/auth/convex-auth>